

ESCOPO E ARQUITETURA - TRIVIA ARENA

1. Escopo do Projeto

O projeto consiste no desenvolvimento de um jogo de trivia multiplayer acessível via navegador. O sistema permite que jogadores se cadastrem ou participem de forma anônima, entrem em salas de jogo criadas por outros usuários e compitam respondendo perguntas de múltipla escolha em tempo real.

Ao acessar a aplicação, o usuário pode se cadastrar informando nome e senha, autenticar-se com uma conta existente ou entrar de forma anônima, caso em que um identificador temporário é gerado no próprio navegador e não persiste entre sessões. Após autenticado ou identificado, o usuário pode criar uma nova sala ou entrar em uma sala existente mediante código compartilhado pelo criador.

Na criação de uma sala, o criador define o número máximo de jogadores entre dois e dez) a quantidade de perguntas da partida (entre cinco e vinte) e o tema das perguntas. Um código único é gerado e deve ser repassado aos demais participantes. A partida só tem início quando o criador a iniciar explicitamente, desde que haja ao menos dois jogadores na sala.

Durante a partida, as perguntas são exibidas a todos os jogadores simultaneamente com suas opções de resposta e um tempo limite por rodada. Cada pergunta distribui um pool de dez créditos entre os jogadores que acertaram, de forma proporcional à ordem de chegada das respostas corretas, quem responde mais rápido recebe uma fatia maior. Ao término de todas as rodadas, um ranking final é exibido com a pontuação acumulada de cada participante. O criador pode iniciar uma nova partida na mesma sala, podendo alterar o tema.

Usuários com cadastro têm acesso a uma página de perfil onde podem atualizar nome e senha, além de consultar um painel de estatísticas históricas contendo número de partidas disputadas, posição média nas partidas, média de pontos, maior pontuação já obtida e número de vitórias. Jogadores anônimos não têm acesso a esse painel, pois seus dados não são persistidos.

2. Visão Geral da Arquitetura

A arquitetura é composta por oito componentes distribuídos que se integram e coordenam para entregar as funcionalidades do sistema. A comunicação entre os componentes utiliza tanto mecanismos síncronos quanto assíncronos, conforme a natureza de cada fluxo. O sistema é projetado para suportar múltiplos clientes concorrentes na Internet e para ser implantado em contêineres Docker sobre instâncias EC2 na AWS.

O sistema é implementado em três linguagens de programação distintas, cada uma alocada onde suas características são mais adequadas: JavaScript no cliente (React), Python no componente de entrada do sistema (API Gateway/FastAPI) e Java nos serviços de domínio que demandam primitivos de concorrência de baixo nível e processamento intensivo (Game Service e User Service).

3. Componentes da Arquitetura

3.1 SPA - Web Client

A interface com o usuário é uma aplicação de página única desenvolvida em React. Ela é responsável por renderizar todas as telas do sistema, cadastro, login, lobby, partida e painel de estatísticas, por manter a comunicação com o backend via dois canais distintos: requisições HTTP REST para operações de gerenciamento e uma conexão WebSocket para toda a comunicação em tempo real durante as partidas. O JWT obtido no login é armazenado em memória e enviado nas chamadas subsequentes que exigem autenticação. Para jogadores anônimos, o identificador temporário é gerado via `crypto.randomUUID()` no próprio navegador, prefixado com `anon:`, e descartado ao fechar a aba.

3.2 API Gateway

O API Gateway é o único ponto de entrada do sistema para os clientes na Internet. É implementado em Python com FastAPI e tem como responsabilidade receber as requisições HTTP REST dos clientes, validar o JWT quando a rota exige autenticação, e traduzir as chamadas para gRPC ao acionar os serviços internos. Também é responsável por fazer proxy das conexões WebSocket dos clientes para o Game Service.

Rotas de jogo, criação e entrada em sala, início de partida e conexão WebSocket, não exigem JWT, permitindo a participação de jogadores anônimos. O Gateway não mantém estado de negócio.

3.3 Game Service

O Game Service é o núcleo do sistema e o principal módulo a ser desenvolvido sob a perspectiva de **concorrência e distribuição**. É implementado em Java com Spring Boot e expõe dois servidores na mesma JVM: um servidor gRPC na porta 9090, que recebe chamadas de gerenciamento de sala vindas do API Gateway, e um servidor WebSocket com STOMP na porta 8080, responsável pela comunicação em tempo real com os clientes durante as partidas.

O serviço gerencia o ciclo de vida completo das salas, que transitam pelos estados de espera, em andamento e finalizada. Ao iniciar uma partida, busca as perguntas do tema selecionado no banco de dados e armazena os identificadores no Redis. Conduz então o fluxo de rodadas, enviando cada pergunta via WebSocket para todos os jogadores conectados à sala e aguardando as respostas dentro de uma janela de tempo configurável.

O processamento concorrente do pool de créditos é o ponto central desse componente. Múltiplas threads recebem as respostas dos jogadores simultaneamente. As respostas corretas de cada rodada são registradas no Redis via **HSETNX**, operação atômica que garante que cada jogador seja contabilizado uma única vez mesmo que múltiplas respostas cheguem concorrentemente ou de instâncias diferentes. Ao encerrar a janela de tempo, a instância condutora da rodada lê todas as respostas com **HGETALL**, ordena por timestamp e distribui os dez créditos proporcionalmente usando a fórmula de pesos decrescentes. O placar é atualizado no Redis de forma atômica via transação **MULTI/EXEC**.

Ao término de cada partida, o Game Service envia o ranking final via WebSocket e publica o evento de resultado no Kafka, de forma assíncrona e independente.

O Game Service pode ser executado em múltiplas instâncias. Como o estado das salas reside integralmente no Redis, qualquer instância pode assumir o atendimento de uma sala em caso de falha de outra. As conexões WebSocket são recuperadas pelos próprios clientes, que detectam a queda via evento **onclose** e reconectam com exponential backoff, ao reconectar, a instância disponível lê o estado da sala no Redis e retoma a partida. Para broadcast entre instâncias, o Redis Pub/Sub é utilizado como barramento interno.

3.4 User Service

O User Service é responsável pelo cadastro e autenticação de usuários e pela gestão de suas estatísticas históricas. É implementado em Java com Spring Boot e expõe um servidor gRPC na porta 9090 para receber chamadas do API Gateway. Gerencia as operações de registro, login, atualização de perfil e consulta de estatísticas.

Além das chamadas síncronas, o User Service opera como consumidor Kafka do tópico game-finished. Ao receber um evento de fim de partida, filtra os jogadores com identificador cadastrado e atualiza as estatísticas históricas de cada um em transação SQL. Esse fluxo é completamente assíncrono e independente do encerramento da partida no Game Service.

3.5 Game State Cache (Redis)

O Redis atua como store de estado das salas ativas, operando em modo master com réplica para disponibilidade. Armazena a configuração e o status de cada sala em estruturas Hash, o placar acumulado dos jogadores e os identificadores das perguntas carregadas para a sessão. O TTL de uma hora nas chaves de sala garante a limpeza automática de salas abandonadas. O Redis também provê o canal de Pub/Sub utilizado para broadcast entre instâncias do Game Service.

3.6 User Database (PostgreSQL)

Banco de dados relacional que persiste os dados de cadastro e as estatísticas históricas dos usuários. Adota o modelo de replicação com uma instância primária e uma réplica de leitura, conectadas via PostgreSQL Streaming Replication. A motivação é a existência de dois workloads com características distintas e incompatíveis: escritas pontuais e transacionais, disparadas pelo consumer Kafka ao fim de cada partida, e leituras contínuas e frequentes das queries do painel de estatísticas. Direcionar cada workload para uma instância dedicada evita que concorram pelo mesmo recurso. A consistência é eventual, aceitável porque o dado, estatística histórica, não impacta decisões imediatas do jogo.

3.7 Question Database (PostgreSQL)

Banco de dados relacional que armazena o acervo de perguntas do sistema. É populado offline antes da operação do sistema via script de carga, podendo ser complementado periodicamente por um processo de ingestão externo.

Adota particionamento horizontal (sharding) entre duas instâncias PostgreSQL fisicamente separadas. Cada instância armazena um subconjunto disjunto de temas: o Shard A contém os temas história, ciência, geografia, artes e literatura e sociedade e cultura; o Shard B contém os temas música, esporte e lazer, filmes e TV, comidas e bebidas e conhecimento geral. Essa separação distribui o acervo de perguntas entre nós distintos, cada um com seu próprio processo de banco de dados, sua própria conexão e seu próprio espaço em disco.

O roteamento é implementado no Game Service, que ao receber o tema da sala resolve o shard correspondente por meio de uma tabela de mapeamento estática e abre a conexão com a instância correta para buscar as perguntas. Se um shard ficar indisponível, partidas com temas daquele shard retornam erro ao criador; partidas com temas do outro shard continuam funcionando normalmente. Essa independência entre shards é a principal vantagem do particionamento horizontal em relação a uma instância monolítica: falhas são isoladas por partição, não propagadas para todo o acervo.

3.8 Message Broker

O Kafka atua como broker de mensagens para o único fluxo genuinamente assíncrono do sistema: a atualização das estatísticas dos jogadores ao término de uma partida. O Game Service publica no tópico game-finished com a chave de partição sendo o código da sala, garantindo ordenação por sala. O User Service consome esse tópico como grupo consumidor dedicado e realiza o commit de offset somente após confirmar a persistência no banco, garantindo at-least-once delivery.

4. Estratégias de Comunicação

O sistema utiliza deliberadamente os dois paradigmas de interação remota exigidos pela especificação do trabalho, cada um aplicado onde é natural.

A comunicação síncrona ocorre em todos os fluxos de gerenciamento: autenticação, cadastro, criação e entrada em sala, início de partida, consulta e edição de perfil. Nesses fluxos, o cliente aguarda a resposta para prosseguir, o que torna o bloqueio adequado. O protocolo utilizado é HTTP REST entre o cliente e o Gateway, e gRPC com Protocol Buffers entre o Gateway e os serviços internos. O gRPC é adotado na comunicação interna por oferecer contratos fortemente tipados via arquivos .proto, serialização binária eficiente e melhor desempenho em chamadas serviço a serviço.

A comunicação assíncrona ocorre em dois contextos. O primeiro é o WebSocket com STOMP, que mantém um canal bidirecional persistente entre o cliente e o Game Service durante toda a partida. Esse canal é assíncrono por natureza: o servidor envia perguntas, resultados de rodada e ranking final sem que o cliente precise solicitá-los, enquanto o cliente envia respostas sem aguardar confirmação imediata. O segundo é o Kafka, que desacopla temporalmente o Game Service do User Service na atualização de estatísticas, o Game Service publica e segue em frente, sem aguardar o processamento downstream.

5. Atendimento às Especificações do Trabalho

O sistema atende às especificações exigidas da seguinte forma:

1. Serviço acessível a múltiplos clientes na Internet: atendido pelo API Gateway como ponto de entrada único e pelo Game Service stateless, cujo estado reside no Redis, permitindo escalonamento horizontal.
2. Integração e coordenação de componentes distribuídos: atendido pela colaboração entre API Gateway, Game Service, User Service, Redis, dois bancos PostgreSQL e Kafka
3. Acessos concorrentes a recursos compartilhados: atendido pelo processamento concorrente do pool de créditos no Game Service: respostas corretas são registradas atomicamente no Redis via **HSETNX** (coordenação entre instâncias), e o placar é atualizado via **MULTI/EXEC** (atomicidade transacional). A operação **HSETNX** garante que acessos concorrentes ao mesmo recurso compartilhado, a resposta de uma rodada, sejam coordenados corretamente mesmo com múltiplas instâncias do serviço.
4. Processamento no servidor concorrentemente com os acessos dos clientes: atendido pelo Game Service, que processa respostas de múltiplos jogadores em paralelo para calcular a distribuição de créditos enquanto outros clientes ainda estão na janela de resposta.
5. Mecanismos de interação síncrona e assíncrona: atendido pelo HTTP REST e gRPC para fluxos síncronos, pelo WebSocket para comunicação em tempo real e pelo Kafka para o fluxo assíncrono de atualização de estatísticas.
6. Replicação e particionamento de dados e funcionalidades: : o particionamento de dados é atendido pelo sharding horizontal do Question Database entre duas instâncias PostgreSQL fisicamente separadas (Shard A e Shard B), com lógica de roteamento por tema implementada no Game Service. A replicação de dados é atendida pelo User Database com instância primária e réplica de leitura. A replicação de funcionalidade é atendida pelo escalonamento horizontal do Game Service em múltiplas instâncias.
7. Consistência e disponibilidade: atendido pelas transações **MULTI/EXEC** no Redis, pelos primitivos de concorrência Java no pool de créditos, pelas transações SQL no User Service e pelas filas duráveis com commit manual de offset no Kafka.

6. Implantação

O sistema é implantado na AWS utilizando instâncias EC2 com Docker. Cada componente é containerizado e orquestrado via Docker Compose. O Game Service é executado em múltiplas réplicas de contêiner com um Nginx configurado como load balancer à frente,

utilizando a diretiva `ip_hash` para sticky sessions, necessário para garantir que o mesmo cliente permaneça conectado à mesma instância durante a sessão WebSocket ativa. As portas definidas nos contratos de cada serviço são portas internas dos contêineres; a exposição externa é gerenciada pelo Application Load Balancer da AWS, transparente para os serviços.